

安徽大学《机器视觉视觉》实验报告（2）

学号： WA2224013 专业： 机器人工程 姓名及签字： 郭义月
实验日期： 2025.5.25 实验成绩： _____ 教师签字： _____

【实验名称】：图像批量读入与处理

【实验目的】：

1. 熟悉和掌握 matlab 基本编程环境
2. 熟悉和掌握基于 matlab 的计算机视觉编程基础
3. 通过 matlab 编程实现图像的批量读入与处理

【实验内容】

案例背景：

在对图像进行处理时，通常是对图像进行图像变换、图像压缩和图像分割等操作。图像处理过程中，往往需要同时对多张图像进行处理，这样可以大大提高工作效率。

批量读入图像：

批量读入图像在 MATLAB 中可以通过循环读入图像实现批量处理。

单个图像处理：

在批量处理图像前，往往需要对单一图像进行处理，预览图像处理结果。

图像分割批量处理：

批量图像往往与单一图像特征非常相似，因此，在对单一图像处理时，设置好最佳参数，可以提高在批量处理时的成功率。

【实验代码和结果】

实验代码 1：批量读入图像：

实验代码：

```
clc;clear;close all;  
% 指定图像文件夹路径  
file_path = '..\实验课数据\图像批处理\';  
% 获取该文件夹中所有 jpg 格式的图像  
img_path_list = dir(strcat(file_path,'*.jpg'));  
img_num = length(img_path_list);  
if img_num > 0  
    for j = 1:img_num  
        img_name = img_path_list(j).name; %逐一读取图像名字  
        image = imread(strcat(file_path, img_name)); %拼接路径  
        subplot(2,3,j);  
        imshow(image);  
        title(['图像', num2str(j)]);  
    end  
end
```

实验结果：

图像1



图像2



图像3



图像4



图像5



图像6



批量读入的另一种方法：使用元胞数组

```
clc;clear; close all;  
% 批量读取图像的一种简单方法：用元胞数组  
file_path = '..\实验课数据\图像批处理\';  
img_path_list = dir(strcat(file_path,'*.jpg'));  
n = length(img_path_list);  
I = cell(1, n);  
for i = 1 : n  
    imageName = img_path_list(i).name;  
    I{i} = imread(strcat(file_path,imageName));  
    subplot(2,3,i);  
    imshow(I{i});  
    title(['图像',num2str(i)]);  
end
```

实验结果：

图像1



图像2



图像3



图像4



图像5



图像6



实验代码 2：图像处理

先对单一图像进行处理，预览图像处理结果，调整好参数之后再对图像进行批量处理

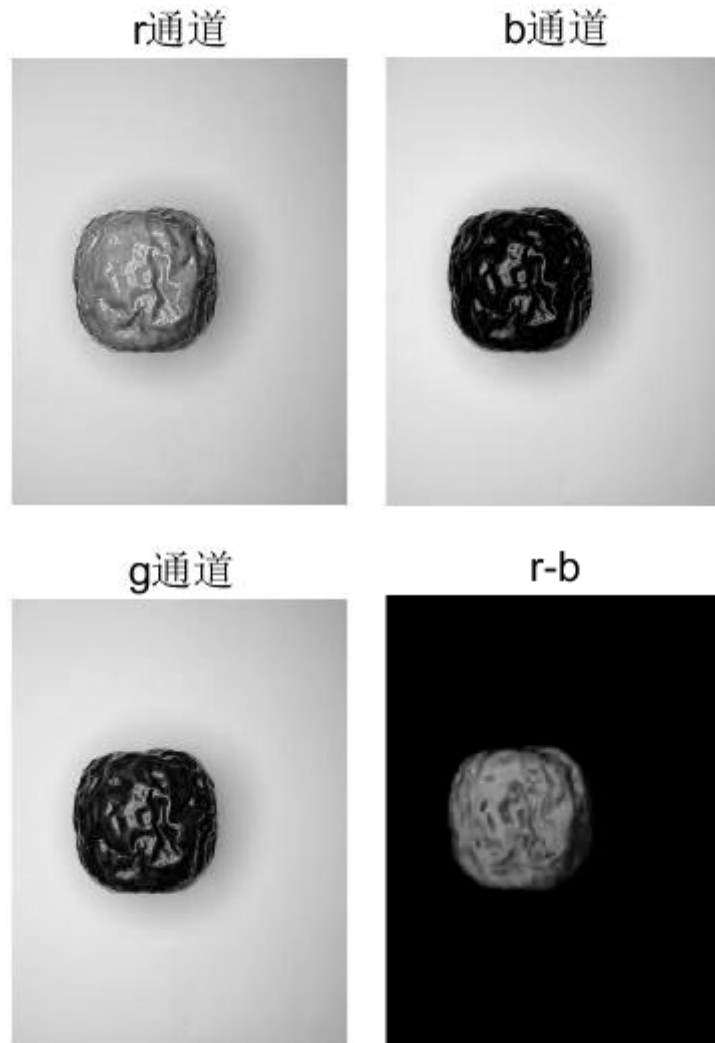
实验代码：

```
clc; clear; close all;  
I = imread('../实验课数据/图像批处理/1.jpg');  
subplot 121; imshow(I); title('初始图像');  
% 取出三个通道  
I_r = I(:,:,1);  
I_g = I(:,:,2);  
I_b = I(:,:,3);  
  
I_rb = I_r - I_b;  
Obj = im2bw(I_rb, 5/255);  
subplot 122; imshow(Obj); title('分割后的图像');
```

实验结果：



中间过程的图片如下图：



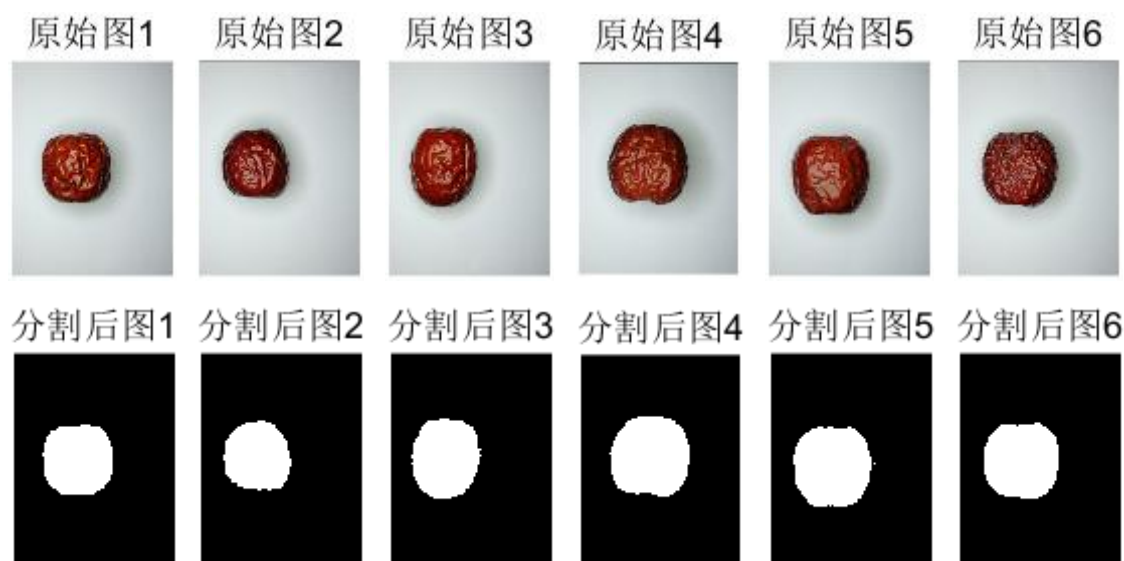
实验代码 3：图像分割批量处理

实验代码：

```
clc;clear;close all;
file_path = '../实验课数据/图像批处理/';
img_path = dir(strcat(file_path,'*.jpg'));
n = length(img_path);
I = cell(1, n);
for i = 1:n
    imageName = img_path(i).name;
    I{i} = imread(strcat(file_path, imageName));
    subplot(2,n,i);imshow(I{i});title(['原始图', num2str(i)]);
    I_r = I{i}(:, :, 1);
    I_g = I{i}(:, :, 2);
    I_b = I{i}(:, :, 3);
    I_rb = I_r - I_b;
    Obj = im2bw(I_rb, 5/255);
```

```
subplot(2,n,i+n);imshow(Obj);title(['分割后图', num2str(i)]);  
end
```

实验结果：



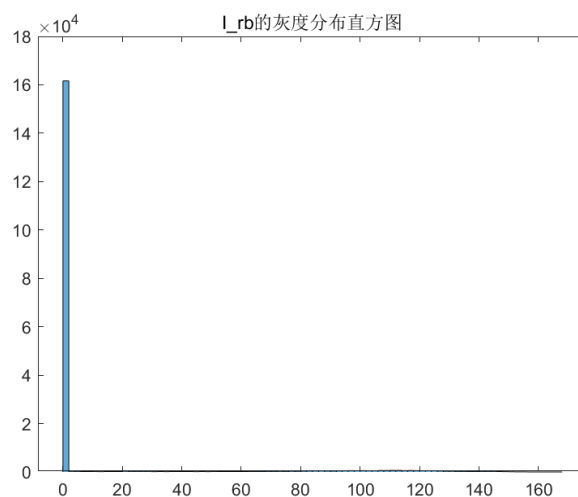
【小结或讨论】

在本次实验中，完成了数据的批量读取和处理，通过使用 for 循环和元胞数组，实现了一次性读取多个图像，并批量处理，这在实际的使用中非常有用。

值得讨论的地方在于，对图像进行分割时参数的调整，重点为什么要进行如下操作：

```
I_rb = I_r - I_b;  
Obj = im2bw(I_rb, 5/255);
```

可以先查看 I_{rb} 的灰度直方图

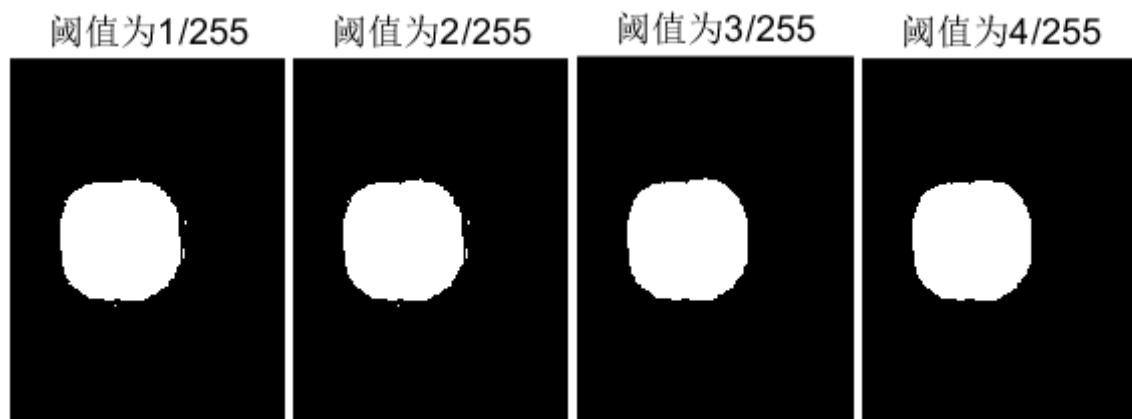


可以观察到，主要都是黑色，其他少了分布在 20-130 之间，此时可以将阈值设置为，5/255，将灰度图变成二值图像时，前 5 个灰度值映射为黑色，后面 250 个灰度值映射到白色。

可以调整参数，比如，分布调整前 1，2，3，4 个灰度值为黑色，代码如下：

```
Obj = im2bw(I_rb, 1/255);  
subplot 141; imshow(Obj); title('阈值为 1/255');  
sum(Obj(:))  
Obj = im2bw(I_rb, 2/255);  
subplot 142; imshow(Obj); title('阈值为 2/255');  
sum(Obj(:))  
Obj = im2bw(I_rb, 3/255);  
subplot 143; imshow(Obj); title('阈值为 3/255');  
sum(Obj(:))  
Obj = im2bw(I_rb, 4/255);  
subplot 144; imshow(Obj); title('阈值为 4/255');  
sum(Obj(:))
```

结果如下图：



可以观察到，随着阈值的调高，白色部分在减少，虽然不明显，但是可以通过对其二值图像求和得到，再二值图像中，0 表示黑色，1 表示白色，所以如果求和的结果越大，代表白色越多，即在设置阈值时，白色部分的占比更多。

分别对 1，2，3，4 求和，结果为：

```
ans =  
  
22873  
ans =
```

```
22796
ans =

22596
ans =

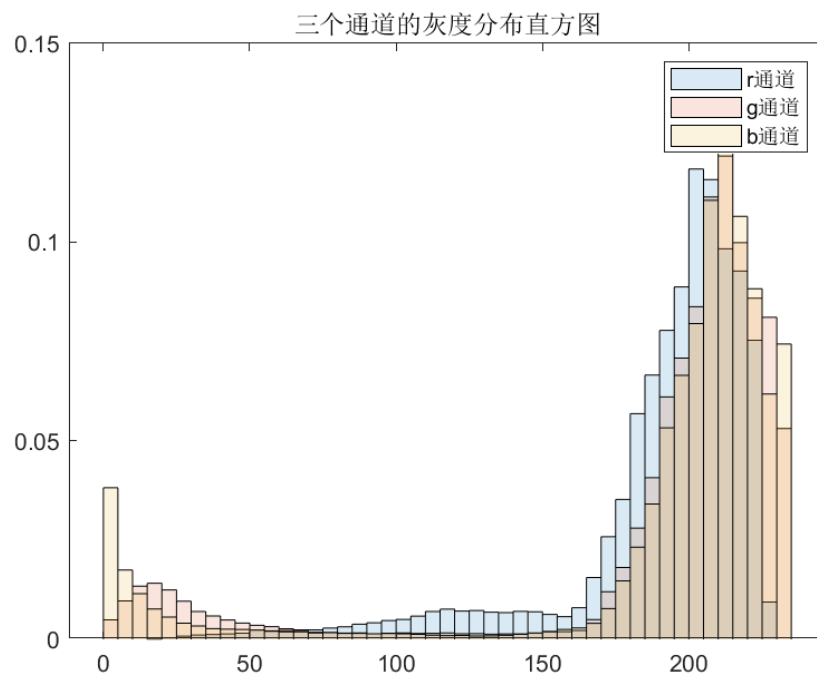
22464
```

求和结果符合猜想。

另外一个值得讨论的问题是，为什么要计算 $I_{rb} = I_r - I_b$

实际上，在本案例中，改成其他的相减也能得到类似的结果，只要看直方图的分布就可以判断这个通道运算完成之后是否适合做阈值分析。

分别显示每个通道的直方图：



可以看出，在本案例中，红绿蓝三个通道的灰度分布几乎是接近的，所以无论哪个通道相减，在调整好阈值后，都可以得到类似的结果。